

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 4
Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size
 F Filter Size
 P Padding
 S Stride

2.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

3. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

4. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

Total trainable parameters = 448.

6. Dataset

Source Code:

https://github.com/Hasini923/Deep_Learning/tree/main/CNN_implementation_for_image_classification

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

7. Effect of Kernel Size on Feature Map Size

A single convolution layer with 8 filters was applied to a sample $32 \times 32 \times 3$ image using kernels of size 3×3 , 5×5 , and 7×7 (stride = 1, no padding).

Kernel Size	Output Feature Map Shape
3×3	$30 \times 30 \times 8$
5×5	$28 \times 28 \times 8$
7×7	$26 \times 26 \times 8$

Inference: As the kernel size increases, the resulting feature map becomes smaller, since a larger filter cannot slide as many times across the input. This confirms the inverse relationship between kernel size and output dimensions.

8. Effect of Stride and Padding

The same convolution layer (kernel = 3×3) was tested with different stride and padding combinations.

Stride	Padding	Output Shape
1	Same	$32 \times 32 \times 8$
1	Valid	$30 \times 30 \times 8$
2	Same	$16 \times 16 \times 8$
2	Valid	$15 \times 15 \times 8$

Inference: Increasing the stride reduces the output size because the filter skips more pixels between steps. "Same" padding preserves the spatial dimensions (or halves them cleanly under stride 2), while "Valid" padding shrinks the output further since no border pixels are added.

9. Comparison between Max Pooling and Average Pooling

Two small CNNs, identical except for the pooling layer, were each trained for 5 epochs.

Comparison Table:

Metric	Max Pooling	Average Pooling
Output Shape after 1st Pool	$16 \times 16 \times 32$	$16 \times 16 \times 32$
Final Validation Accuracy (5 epochs)	0.6942	0.6742

Inference: Both pooling methods reduce the spatial dimensions identically, but Max Pooling achieved a higher validation accuracy than Average Pooling under the same 5-epoch training budget, suggesting it retains the more discriminative activations for this dataset.

10. Mandatory Plots

10.1 Sample Images

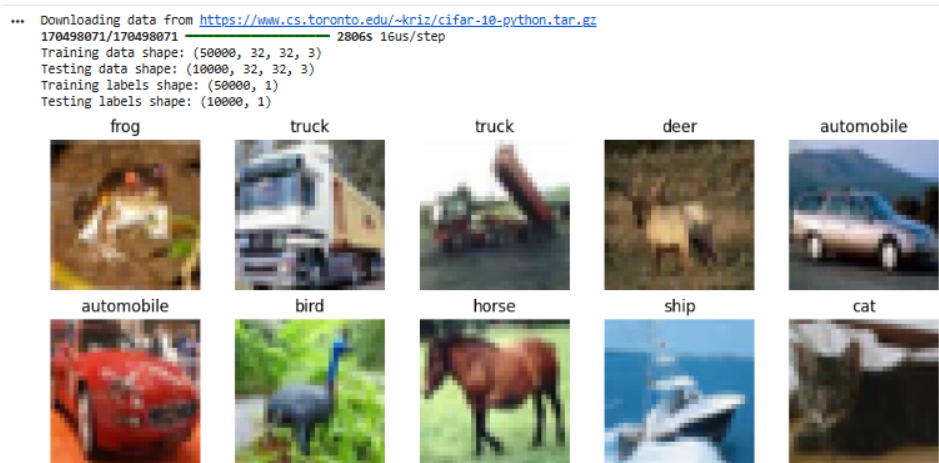


Figure 1: Sample images from the CIFAR-10 dataset.

Interpretation: The images show different classes from the CIFAR-10 dataset, such as frog, truck, deer, automobile, bird, horse, ship, and cat. The images are small and have low resolution, but the objects are still recognizable based on their visual features.

10.2 Class Distribution

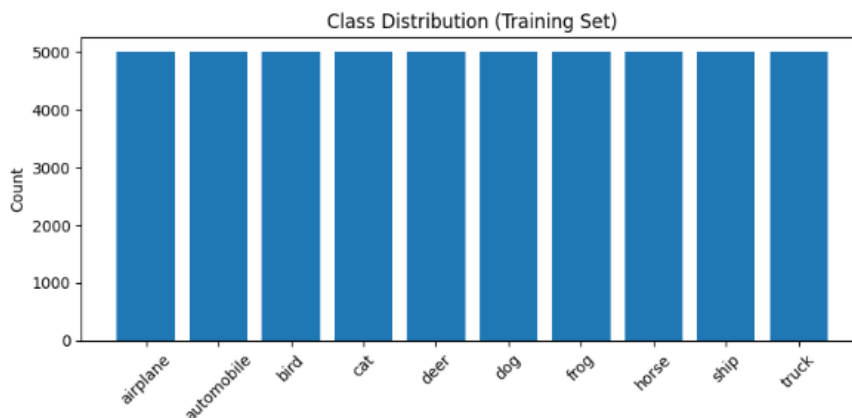


Figure 2: Distribution of classes in the training set.

Interpretation: The class distribution is exactly equal for all 10 CIFAR-10 classes, with 5,000 images in each class. This confirms that the dataset is perfectly balanced and does not require any class-imbalance handling.

10.3 Feature Maps

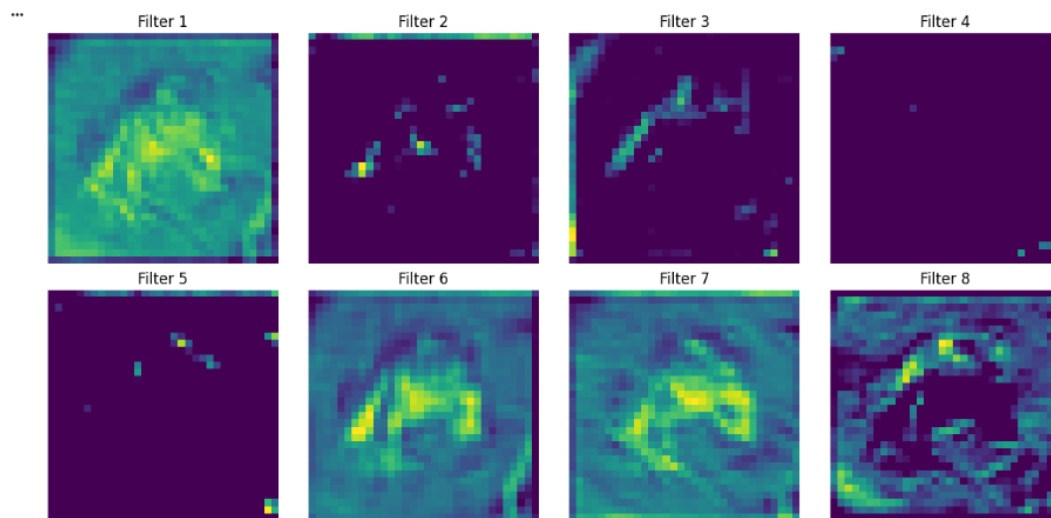


Figure 3: Feature maps from the first convolution layer (16 filters, showing 8).

Interpretation: Different filters respond to different visual structures in the input image; some filters highlight the object's outline while others activate only on small localized regions. This shows how early convolution filters begin extracting distinct low-level features from the same image.

10.4 Training Accuracy

...	conv2d_16 (Conv2D)	(None, 32, 32, 32)	896
	max_pooling2d_4 (MaxPooling2D)	(None, 16, 16, 32)	0
	conv2d_17 (Conv2D)	(None, 16, 16, 64)	18,496
	max_pooling2d_5 (MaxPooling2D)	(None, 8, 8, 64)	0
	flatten_4 (Flatten)	(None, 4096)	0
	dense_8 (Dense)	(None, 128)	524,416
	dense_9 (Dense)	(None, 10)	1,290

Total params: 545,098 (2.08 MB)
 Trainable params: 545,098 (2.08 MB)
 Non-trainable params: 0 (0.00 B)

Epoch 1/20
 1563/1563 ————— 97s 61ms/step - accuracy: 0.5163 - loss: 1.3545 - val_accuracy: 0.6036 - val_loss: 1.1111
 Epoch 2/20
 1563/1563 ————— 94s 60ms/step - accuracy: 0.6566 - loss: 0.9814 - val_accuracy: 0.6698 - val_loss: 0.9353
 Epoch 3/20
 1563/1563 ————— 92s 59ms/step - accuracy: 0.7053 - loss: 0.8397 - val_accuracy: 0.6904 - val_loss: 0.8868
 Epoch 4/20
 1563/1563 ————— 146s 62ms/step - accuracy: 0.7402 - loss: 0.7397 - val_accuracy: 0.6841 - val_loss: 0.9128
 Epoch 5/20
 1563/1563 ————— 137s 59ms/step - accuracy: 0.7722 - loss: 0.6479 - val_accuracy: 0.7064 - val_loss: 0.8532
 Epoch 6/20
 1563/1563 ————— 96s 61ms/step - accuracy: 0.7993 - loss: 0.5721 - val_accuracy: 0.7081 - val_loss: 0.8771
 Epoch 7/20
 1563/1563 ————— 139s 59ms/step - accuracy: 0.8223 - loss: 0.5016 - val_accuracy: 0.7128 - val_loss: 0.9057
 Epoch 8/20
 1563/1563 ————— 92s 59ms/step - accuracy: 0.8482 - loss: 0.4286 - val_accuracy: 0.7166 - val_loss: 0.9462
 Epoch 9/20
 1563/1563 ————— 92s 59ms/step - accuracy: 0.8714 - loss: 0.3672 - val_accuracy: 0.7136 - val_loss: 0.9759
 Epoch 10/20
 1563/1563 ————— 93s 59ms/step - accuracy: 0.8910 - loss: 0.3072 - val_accuracy: 0.7070 - val_loss: 1.1030
 Epoch 11/20
 1563/1563 ————— 142s 60ms/step - accuracy: 0.9077 - loss: 0.2595 - val_accuracy: 0.7025 - val_loss: 1.1400
 Epoch 12/20
 1563/1563 ————— 93s 59ms/step - accuracy: 0.9209 - loss: 0.2185 - val_accuracy: 0.7071 - val_loss: 1.2471
 Epoch 13/20
 1563/1563 ————— 93s 60ms/step - accuracy: 0.9331 - loss: 0.1855 - val_accuracy: 0.6990 - val_loss: 1.3156
 Epoch 14/20
 1563/1563 ————— 141s 59ms/step - accuracy: 0.9458 - loss: 0.1534 - val_accuracy: 0.7013 - val_loss: 1.4850
 Epoch 15/20
 1563/1563 ————— 93s 59ms/step - accuracy: 0.9521 - loss: 0.1364 - val_accuracy: 0.7061 - val_loss: 1.5526
 Epoch 16/20
 1563/1563 ————— 142s 59ms/step - accuracy: 0.9576 - loss: 0.1179 - val_accuracy: 0.6970 - val_loss: 1.6841
 Epoch 17/20
 1563/1563 ————— 93s 60ms/step - accuracy: 0.9600 - loss: 0.1123 - val_accuracy: 0.6987 - val_loss: 1.8044
 Epoch 18/20
 1563/1563 ————— 145s 62ms/step - accuracy: 0.9663 - loss: 0.0953 - val_accuracy: 0.6985 - val_loss: 1.9165
 Epoch 19/20
 1563/1563 ————— 138s 59ms/step - accuracy: 0.9665 - loss: 0.0959 - val_accuracy: 0.6880 - val_loss: 1.9449
 Epoch 20/20
 1563/1563 ————— 143s 60ms/step - accuracy: 0.9704 - loss: 0.0832 - val_accuracy: 0.6974 - val_loss: 2.0923

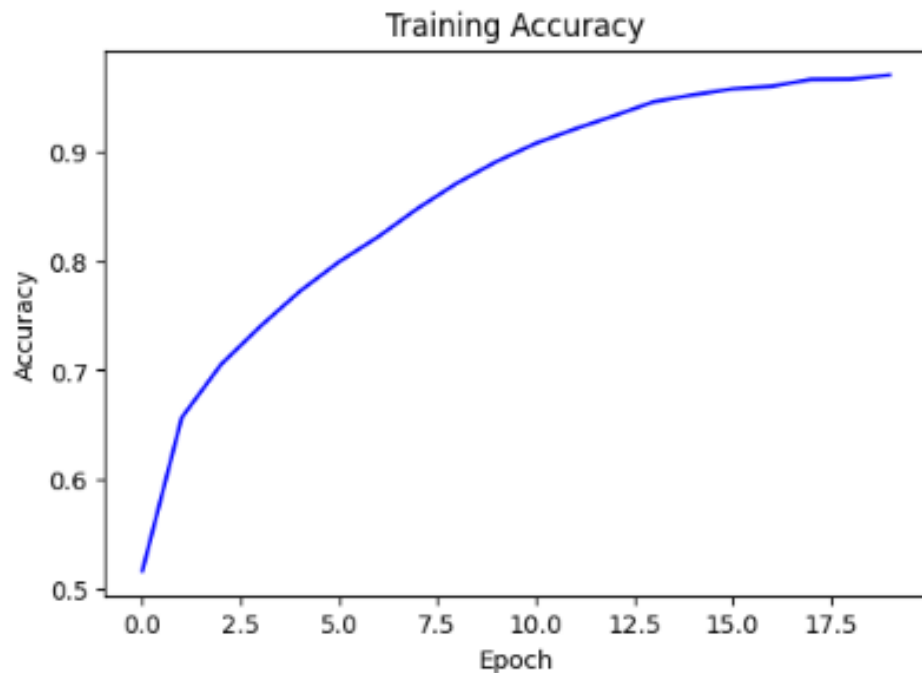


Figure 4: Training accuracy over epochs.

Interpretation: The training accuracy rises steadily throughout all 20 epochs, reaching around 97% by the final epoch. This shows the model is fitting the training data increasingly well as training progresses.

10.5 Validation Accuracy

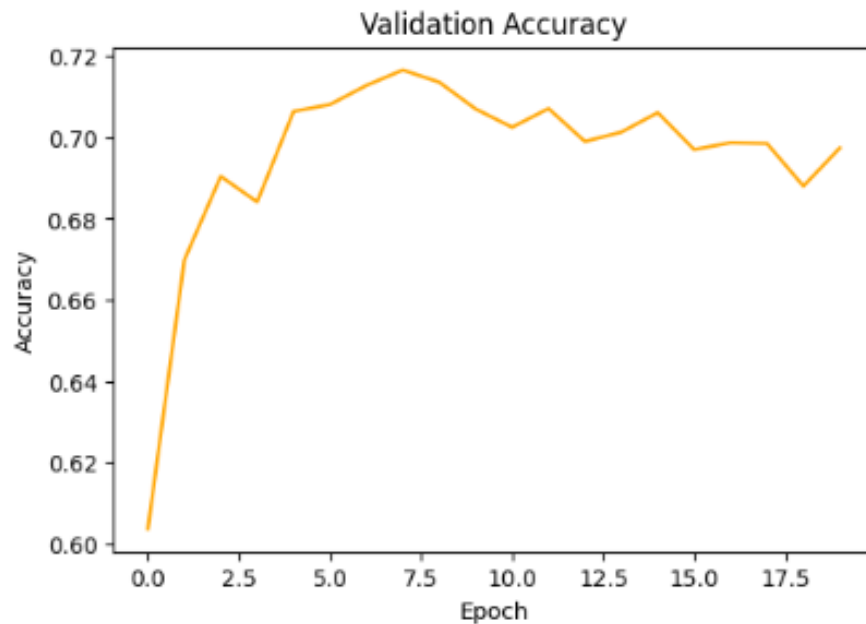


Figure 5: Validation accuracy over epochs.

Interpretation: The validation accuracy climbs quickly in the first few epochs and peaks around epoch 7-8 near 71–72%. After that it plateaus and fluctuates slightly, indicating the model has stopped generalizing better on unseen data.

10.6 Training Loss

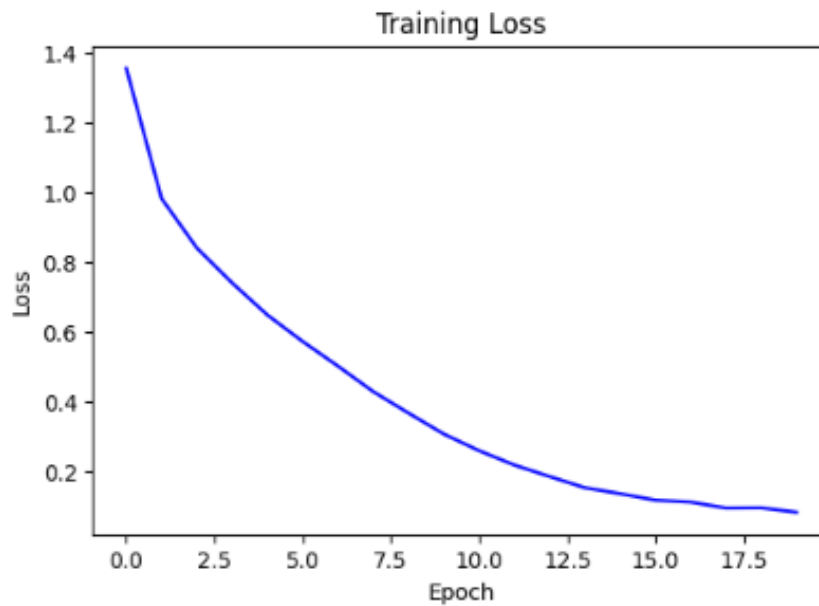


Figure 6: Training loss over epochs.

Inference: The training loss decreases consistently across all epochs, dropping close to 0.08 by the end. This shows the model is continuing to fit the training set more closely with each epoch.

10.7 Validation Loss

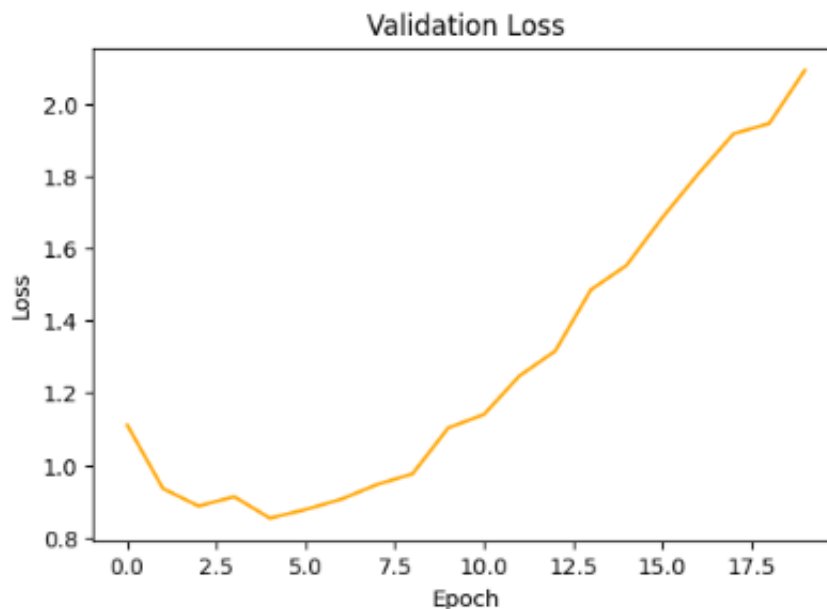


Figure 7: Validation loss over epochs.

Interpretation: The validation loss reaches its lowest point around epoch 4-5 and then rises steadily for the rest of training, while training loss keeps falling. This growing gap is a clear

sign of overfitting in the later epochs.

10.8 Confusion Matrix

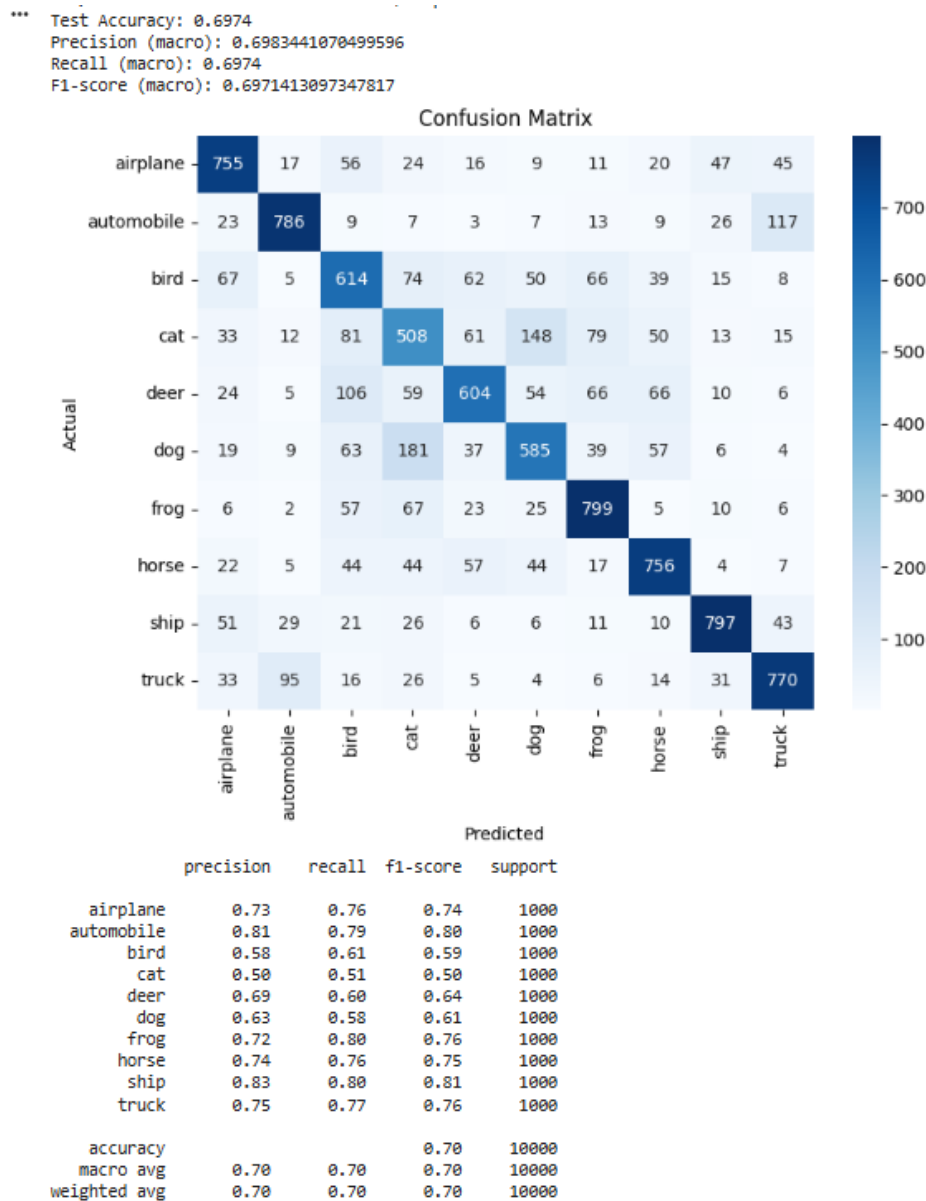


Figure 8: Confusion matrix of the evaluated model.

Interpretation: The model performs strongly on classes like Automobile, Frog, Ship, and Horse, with high diagonal counts. The most notable confusion occurs between Cat and Dog, and between Automobile and Truck, which are visually and semantically similar categories.

11. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**
3. **Compare ReLU and Sigmoid activation functions.**
4. **Replace Max Pooling with Average Pooling and compare the results.**
5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

12. Results

The evaluation metrics for the primary Convolutional Neural Network (Max Pooling model, 20 epochs) are recorded below:

- **Training Accuracy:** 97.04%
- **Testing Accuracy:** 69.74%
- **Precision (macro):** 0.6983441070499596
- **Recall (macro):** 0.6974
- **F1-score (macro):** 0.6971413097347817
- **Number of Parameters:** 545,098

13. Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?**
Convolution preserves the spatial structure of images and detects local features like edges and textures using shared weights. It also uses far fewer parameters than fully connected layers for the same input size.
2. **How does stride affect the feature map size?**
A larger stride moves the filter by more pixels, resulting in a smaller feature map. A smaller stride produces a larger, more finely-sampled feature map.
3. **What is the role of padding?**
Padding adds extra pixels around the image to control the output size. It also helps preserve information from the edges of the image that would otherwise be under-represented.
4. **Why is pooling used?**
Pooling reduces the spatial dimensions of feature maps, which decreases computation and the number of parameters downstream. It also helps the model focus on the most important features while adding some translation invariance.
5. **How do feature maps represent image characteristics?**
Feature maps show the regions of the image that activate each convolution filter, such as edges, shapes, textures, and patterns. Different filters detect different characteristics of the same image.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs use shared weights and local connections, so the same small filter is applied across the entire image instead of connecting every input pixel to every neuron. This greatly reduces the number of trainable parameters compared to MLPs.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.